LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Event Ticket Tier Processor

Name: Rekha Dhorigol

SRN: PES1UG24CS370

Section: G

1. Objective

The objective of this experiment is to perform fuzz testing on an Event Ticket Tier Processor application using the Hypothesis Python library.

The experiment aims to:

- Identify input-validation bugs by generating unexpected and malformed inputs.
- Test the robustness of application functions against arbitrary input values.
- Identify cases where the application crashes due to invalid input.
- Improve the application by implementing appropriate input validation and error handling.
- Verify that the corrected application handles unexpected inputs safely.

2. Software and tools used

- Programming Language: Python
- IDE: Visual Studio Code
- Testing Framework: Pytest
- Fuzz Testing Library: Hypothesis
- Version Control: Git and GitHub
- Operating System: Windows 11

The required Python packages were installed using:

python -m pip install hypothesis pytest

3. Application description

The Event Ticket Tier Processor consists of 3 main functions:

1. **decode_promo_voucher():** Processes promotional voucher codes and extracts the discount percentage.
2. **calculate_seat_tier_price():** Calculates the ticket price based on the seat category and row number.

3. **parse_attendee_badge():** Processes attendee information containing the attendee's name, age, and seat number.

The original implementation assumed that the user would always provide correctly formatted input. It did not adequately validate malformed or unexpected inputs, which could result in runtime exceptions.

4. Execution of the original application

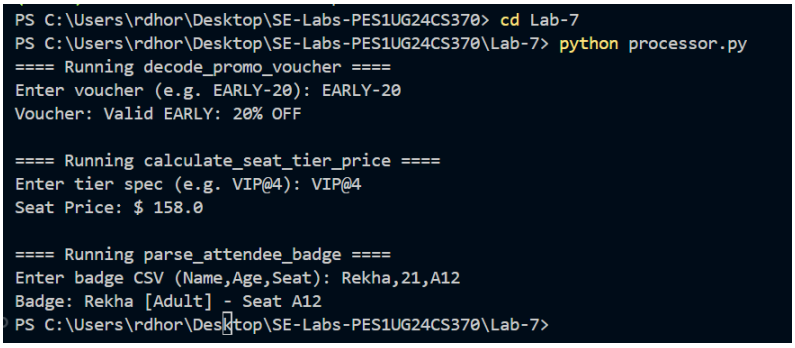
The original application was executed before making any modifications.
The following valid inputs were provided:

- Voucher Code: EARLY-20
- Seat Tier Specification: VIP@4
- Attendee Details: Rekha,21,A12

The application produced the expected results:

- Voucher: Valid EARLY: 20% OFF
- Seat Price: $158.0
- Badge: Rekha [Adult] - Seat A12

S1: Execution of the Original Application



5. Implementation of hypothesis fuzz tests

Hypothesis is a property-based testing library for Python. It automatically generates a wide range of input values to test whether a program behaves correctly.
The test cases were implemented in test_processor.py using the @given decorator and Hypothesis strategies.
The following strategy was used:
@given(st.text() | st.none())
This strategy generates arbitrary strings, including empty strings and unexpected text, as well as None values.
Three fuzz test functions were implemented:

1. test_decode_promo_voucher_no_crash()
2. test_calculate_seat_tier_price_safe()

3. test_parse_attendee_badge_safe()

Each test invokes its respective application function inside a try-except block. If an exception occurs, the test fails and reports the input that caused the failure.
This helps identify unexpected input-validation issues without manually writing every possible test input.

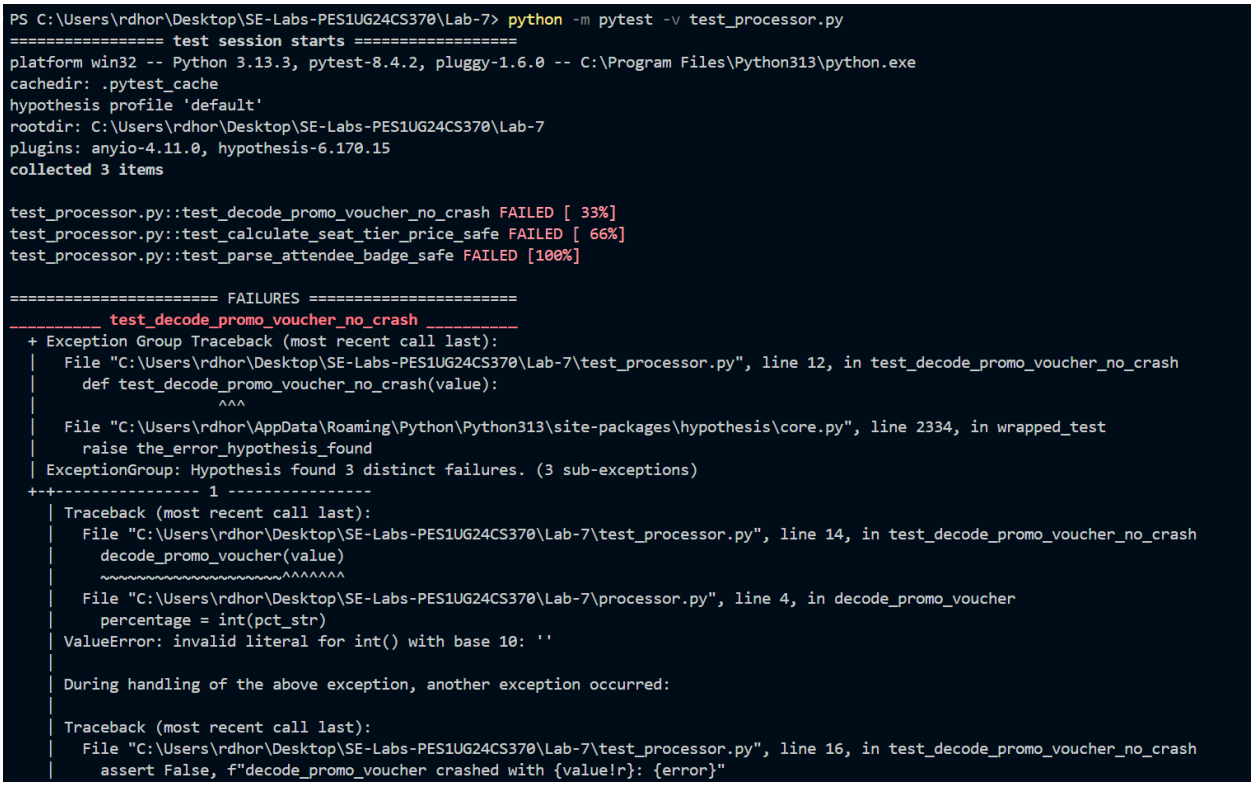
6. Fuzz testing before fixing the application

The Hypothesis tests were executed against the original implementation using:
python -m pytest -v test_processor.py
The generated inputs exposed input-handling problems in the original functions.
Examples of potential failure conditions included:

- None values being passed to string operations.
- Voucher codes with missing or extra separators.
- Invalid percentage values.
- Seat specifications with missing or invalid row numbers.
- Unknown seat categories.
- Attendee details with missing fields or invalid age values.

The original implementation did not handle all these cases safely, resulting in exceptions during fuzz testing.

S2: Hypothesis Fuzz Testing Before Fixing



```
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7> python -m pytest -v test_processor.py
================= test session starts ==================
platform win32 -- Python 3.13.3, pytest-8.4.2, pluggy-1.6.0 -- C:\Program Files\Python313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7
plugins: anyio-4.11.0, hypothesis-6.170.15
collected 3 items

test_processor.py::test_decode_promo_voucher_no_crash FAILED [ 33%]
test_processor.py::test_calculate_seat_tier_price_safe FAILED [ 66%]
test_processor.py::test_parse_attendee_badge_safe FAILED [100%]

====================== FAILURES ======================
_________ test_decode_promo_voucher_no_crash _________
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 12, in test_decode_promo_voucher_no_crash
  |     def test_decode_promo_voucher_no_crash(value):
  |                    ^^^
  |   File "C:\Users\rdhor\AppData\Roaming\Python\Python313\site-packages\hypothesis\core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 14, in test_decode_promo_voucher_no_crash
    |     decode_promo_voucher(value)
    |     ~~~~~~~~~~~~~~~~~~~~^^^^^^^
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\processor.py", line 4, in decode_promo_voucher
    |     percentage = int(pct_str)
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 16, in test_decode_promo_voucher_no_crash
    |     assert False, f"decode_promo_voucher crashed with {value!r}: {error}"
```

```
  | AssertionError: decode_promo_voucher crashed with '-': invalid literal for int() with base 10: ''
  | assert False
  | Failing test case: test_decode_promo_voucher_no_crash(
  |     value='-',
  | )
  +---------------- 2 ----------------
  | Traceback (most recent call last):
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 14, in test_decode_promo_voucher_no_crash
  |     decode_promo_voucher(value)
  |     ~~~~~~~~~~~~~~~~~~~~^^^^^^^
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\processor.py", line 3, in decode_promo_voucher
  |     prefix, pct_str = code.split('-')
  |     ^^^^^^^^^^^^^^
  | ValueError: not enough values to unpack (expected 2, got 1)
  |
  | During handling of the above exception, another exception occurred:
  |
  | Traceback (most recent call last):
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 16, in test_decode_promo_voucher_no_crash
  |     assert False, f"decode_promo_voucher crashed with {value!r}: {error}"
  | AssertionError: decode_promo_voucher crashed with '': not enough values to unpack (expected 2, got 1)
  | assert False
  | Failing test case: test_decode_promo_voucher_no_crash(
  |     value='',  # or any other generated value
  | )
  +---------------- 3 ----------------
  | Traceback (most recent call last):
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 14, in test_decode_promo_voucher_no_crash
  |     decode_promo_voucher(value)
  |     ~~~~~~~~~~~~~~~~~~~~^^^^^^^
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\processor.py", line 2, in decode_promo_voucher
  |     code = voucher_code.strip()
  |            ^^^^^^^^^^^^^^^^^
  | AttributeError: 'NoneType' object has no attribute 'strip'
  |
  | During handling of the above exception, another exception occurred:
```

```
  | Traceback (most recent call last):
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 16, in test_decode_promo_voucher_no_crash
  |     assert False, f"decode_promo_voucher crashed with {value!r}: {error}"
  | AssertionError: decode_promo_voucher crashed with None: 'NoneType' object has no attribute 'strip'
  | assert False
  | Failing test case: test_decode_promo_voucher_no_crash(
  |     value=None,  # or any other generated value
  | )
  +-----------------------------------
_________ test_calculate_seat_tier_price_safe _________
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 20, in test_calculate_seat_tier_price_safe
  |     def test_calculate_seat_tier_price_safe(value):
  |                   ^^^
  |   File "C:\Users\rdhor\AppData\Roaming\Python\Python313\site-packages\hypothesis\core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 22, in test_calculate_seat_tier_price_safe
    |     calculate_seat_tier_price(value)
    |     ~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\processor.py", line 10, in calculate_seat_tier_price
    |     row = int(parts[1])
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 24, in test_calculate_seat_tier_price_safe
    |     assert False, f"calculate_seat_tier_price crashed with {value!r}: {error}"
    | AssertionError: calculate_seat_tier_price crashed with '@': invalid literal for int() with base 10: ''
    | assert False
    | Failing test case: test_calculate_seat_tier_price_safe(
    |     value='@',
    | )
```

```
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 22, in test_calculate_seat_tier_price_safe
    |     calculate_seat_tier_price(value)
    |     ~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\processor.py", line 10, in calculate_seat_tier_price
    |     row = int(parts[1])
    |               ~~~~~^^^
    | IndexError: list index out of range
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 24, in test_calculate_seat_tier_price_safe
    |     assert False, f"calculate_seat_tier_price crashed with {value!r}: {error}"
    | AssertionError: calculate_seat_tier_price crashed with '': list index out of range
    | assert False
    | Failing test case: test_calculate_seat_tier_price_safe(
    |     value='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 22, in test_calculate_seat_tier_price_safe
    |     calculate_seat_tier_price(value)
    |     ~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\processor.py", line 8, in calculate_seat_tier_price
    |     parts = tier_spec.split('@')
    |             ^^^^^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 24, in test_calculate_seat_tier_price_safe
    |     assert False, f"calculate_seat_tier_price crashed with {value!r}: {error}"
    | AssertionError: calculate_seat_tier_price crashed with None: 'NoneType' object has no attribute 'split'
```

```
    | assert False
    | Failing test case: test_calculate_seat_tier_price_safe(
    |     value=None,  # or any other generated value
    | )
    +------------------------------------
_____________ test_parse_attendee_badge_safe _____________
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 28, in test_parse_attendee_badge_safe
  |     def test_parse_attendee_badge_safe(value):
  |                    ^^^
  |   File "C:\Users\rdhor\AppData\Roaming\Python\Python313\site-packages\hypothesis\core.py", line 2334, in wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 30, in test_parse_attendee_badge_safe
    |     parse_attendee_badge(value)
    |     ~~~~~~~~~~~~~~~~~~~~^^^^^^^
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\processor.py", line 22, in parse_attendee_badge
    |     age = int(tokens[1])
    | ValueError: invalid literal for int() with base 10: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 32, in test_parse_attendee_badge_safe
    |     assert False, f"parse_attendee_badge crashed with {value!r}: {error}"
    | AssertionError: parse_attendee_badge crashed with ',': invalid literal for int() with base 10: ''
    | assert False
    | Failing test case: test_parse_attendee_badge_safe(
    |     value=',',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 30, in test_parse_attendee_badge_safe
    |     parse_attendee_badge(value)
```

```
  |     parse_attendee_badge(value)
  |     ~~~~~~~~~~~~~~~~~~~~~~^^^^^^^
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\processor.py", line 22, in parse_attendee_badge
  |     age = int(tokens[1])
  |               ~~~~~~^^^
  | IndexError: list index out of range
  |
  | During handling of the above exception, another exception occurred:
  |
  | Traceback (most recent call last):
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 32, in test_parse_attendee_badge_safe
  |     assert False, f"parse_attendee_badge crashed with {value!r}: {error}"
  | AssertionError: parse_attendee_badge crashed with '': list index out of range
  | assert False
  | Failing test case: test_parse_attendee_badge_safe(
  |     value='',  # or any other generated value
  | )
  +---------------- 3 ----------------
  | Traceback (most recent call last):
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 30, in test_parse_attendee_badge_safe
  |     parse_attendee_badge(value)
  |     ~~~~~~~~~~~~~~~~~~~~~~^^^^^^^
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\processor.py", line 20, in parse_attendee_badge
  |     tokens = badge_csv.split(',')
  |              ^^^^^^^^^^^^^^^
  | AttributeError: 'NoneType' object has no attribute 'split'
  |
  | During handling of the above exception, another exception occurred:
  |
  | Traceback (most recent call last):
  |   File "C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7\test_processor.py", line 32, in test_parse_attendee_badge_safe
  |     assert False, f"parse_attendee_badge crashed with {value!r}: {error}"
  | AssertionError: parse_attendee_badge crashed with None: 'NoneType' object has no attribute 'split'
  | assert False
  | Failing test case: test_parse_attendee_badge_safe(
  |     value=None,  # or any other generated value
```

```
      |     value=None,  # or any other generated value
      | )
      +-----------------------------------
  =============== short test summary info ================
  FAILED test_processor.py::test_decode_promo_voucher_no_crash - ExceptionGroup: Hypothesis found 3 distinct failures...
  FAILED test_processor.py::test_calculate_seat_tier_price_safe - ExceptionGroup: Hypothesis found 3 distinct failures...
  FAILED test_processor.py::test_parse_attendee_badge_safe - ExceptionGroup: Hypothesis found 3 distinct failures...
  ================= 3 failed in 4.14s ==================
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7>
```

7. Fixes implemented in the application

The original processor.py file was modified to improve input validation & prevent unexpected runtime exceptions.

The following improvements were implemented:

7.1 Promotional Voucher Validation

- Checked whether the input is a string.
- Verified that the voucher contains exactly one hyphen.
- Checked that both the prefix and percentage are present.
- Handled invalid integer conversion.
- Restricted the percentage to the range 0–100.
- Returned an appropriate invalid voucher message for invalid inputs.

7.2 Seat Tier Price Validation

- Checked whether the input is a string.

- Verified that the seat specification contains exactly one @ separator.
- Validated the seat category against the supported categories: VIP, PREMIUM, and GENERAL.
- Handled invalid row number conversion.
- Rejected empty & negative row values.
- Returned None for invalid seat specifications instead of allowing an exception to occur.

7.3 Attendee Badge Validation

- Checked whether the input is a string.
- Verified that exactly three comma-separated fields are provided.
- Checked that the name, age, and seat fields are not empty.
- Handled invalid age conversion.
- Rejected negative age values.
- Returned an appropriate invalid attendee badge message for invalid inputs.

These modifications made the application more robust when processing unexpected or malformed input.

8. Execution of the fixed application

After implementing the validation changes, the application was executed again using the same valid sample inputs:

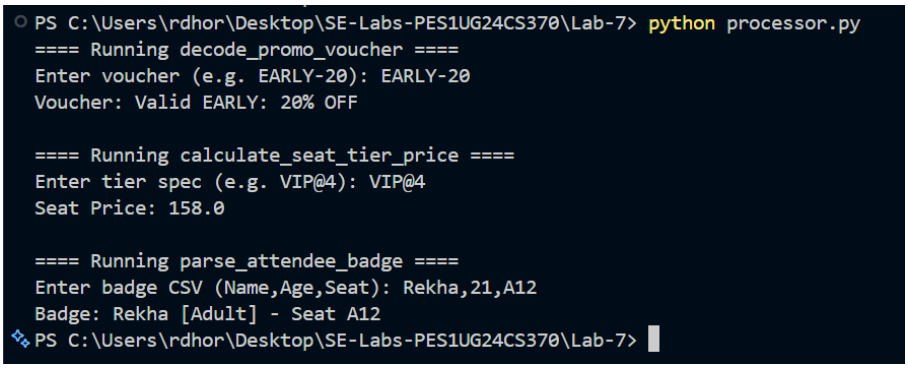
- Voucher Code: EARLY-20
- Seat Tier Specification: VIP@4
- Attendee Details: Rekha,21,A12

The application continued to produce the expected results:

- Voucher: Valid EARLY: 20% OFF
- Seat Price: $158.0
- Badge: Rekha [Adult] - Seat A12

This confirmed that the validation improvements did not affect the expected behaviour for valid inputs.

S3: Execution of the Fixed Application



9. Fuzz testing after fixing the application

The Hypothesis fuzz tests were executed again against the modified application using:
python -m pytest -v test_processor.py
All three fuzz test functions passed successfully.

Test Results:

Test Function	Result
test_decode_promo_voucher_no_crash	PASSED
test_calculate_seat_tier_price_safe	PASSED
test_parse_attendee_badge_safe	PASSED

Total Tests: 3
Passed: 3
Failed: 0
The successful execution indicates that the application handled the generated test inputs without raising unexpected exceptions.

S4: Hypothesis Fuzz Testing After Fixing

```
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7> python -m pytest -v test_processor.py
============================= test session starts =============================
platform win32 -- Python 3.13.3, pytest-8.4.2, pluggy-1.6.0 -- C:\Program Files\Python313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7
plugins: anyio-4.11.0, hypothesis-6.170.15
collected 3 items

test_processor.py::test_decode_promo_voucher_no_crash PASSED             [ 33%]
test_processor.py::test_calculate_seat_tier_price_safe PASSED            [ 66%]
test_processor.py::test_parse_attendee_badge_safe PASSED                 [100%]

============================== 3 passed in 2.72s ==============================
PS C:\Users\rdhor\Desktop\SE-Labs-PES1UG24CS370\Lab-7>
```

10. Git commits & python source codes

1)test_processor.py:

```
from hypothesis import given, strategies as st

from processor import (
    decode_promo_voucher,
    calculate_seat_tier_price,
    parse_attendee_badge,
)


@given(st.text() | st.none())
def test_decode_promo_voucher_no_crash(value):
    try:
```

```
    decode_promo_voucher(value)
  except Exception as error:
    assert False, f"decode_promo_voucher crashed with {value!r}: {error}"


@given(st.text() | st.none())
def test_calculate_seat_tier_price_safe(value):
  try:
    calculate_seat_tier_price(value)
  except Exception as error:
    assert False, f"calculate_seat_tier_price crashed with {value!r}: {error}"


@given(st.text() | st.none())
def test_parse_attendee_badge_safe(value):
  try:
    parse_attendee_badge(value)
  except Exception as error:
    assert False, f"parse_attendee_badge crashed with {value!r}: {error}"
```

2)processor.py:

```
def decode_promo_voucher(voucher_code):
  if not isinstance(voucher_code, str):
    return "Invalid voucher code"

  code = voucher_code.strip()

  if code.count("-") != 1:
    return "Invalid voucher code"

  prefix, pct_str = code.split("-", 1)
  prefix = prefix.strip()

  if not prefix or not pct_str.strip():
    return "Invalid voucher code"

  try:
```

```
        percentage = int(pct_str.strip())
    except ValueError:
        return "Invalid voucher code"

    if not 0 <= percentage <= 100:
        return "Invalid voucher code"

    return f"Valid {prefix.upper()}: {percentage}% OFF"

def calculate_seat_tier_price(tier_spec):
    if not isinstance(tier_spec, str):
        return None

    parts = tier_spec.split("@")

    if len(parts) != 2:
        return None

    tier = parts[0].strip().upper()
    row_text = parts[1].strip()

    tier_rates = {
        "VIP": 150.0,
        "PREMIUM": 90.0,
        "GENERAL": 45.0
    }

    if tier not in tier_rates or not row_text:
        return None

    try:
        row = int(row_text)
    except ValueError:
        return None

    if row < 0:
        return None
```

```
    return tier_rates[tier] + (row * 2.0)

def parse_attendee_badge(badge_csv):
    if not isinstance(badge_csv, str):
        return "Invalid attendee badge"

    tokens = badge_csv.split(",")

    if len(tokens) != 3:
        return "Invalid attendee badge"

    name = tokens[0].strip()
    age_text = tokens[1].strip()
    seat = tokens[2].strip()

    if not name or not age_text or not seat:
        return "Invalid attendee badge"

    try:
        age = int(age_text)
    except ValueError:
        return "Invalid attendee badge"

    if age < 0:
        return "Invalid attendee badge"

    category = "Child" if age < 18 else "Adult"

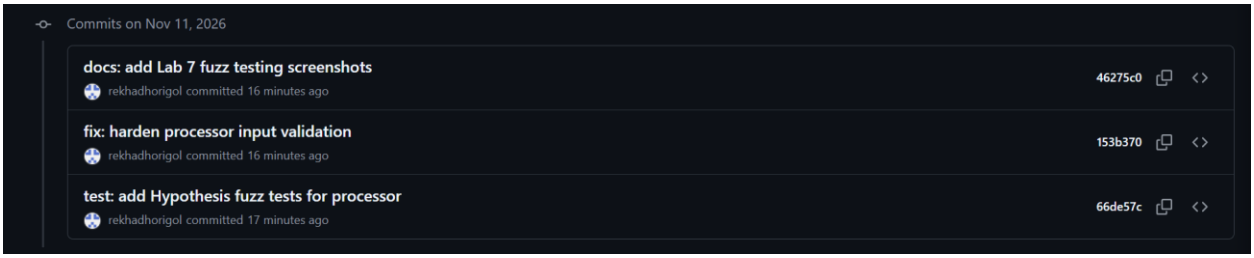
    return f"{name} [{category}] - Seat {seat}"

def main():
    print("==== Running decode_promo_voucher ====")
    v_input = input("Enter voucher (e.g. EARLY-20): ")
    print("Voucher:", decode_promo_voucher(v_input))

    print("\n==== Running calculate_seat_tier_price ====")
    t_input = input("Enter tier spec (e.g. VIP@4): ")
    print("Seat Price:", calculate_seat_tier_price(t_input))
```

```
    print("\n==== Running parse_attendee_badge ====")
    b_input = input("Enter badge CSV (Name,Age,Seat): ")
    print("Badge:", parse_attendee_badge(b_input))

if __name__ == "__main__":
    main()
```



GitHub Repository:
https://github.com/rekhadhorigol/SE-Labs-PES1UG24CS370

11. Conclusion

In this experiment, fuzz testing was performed on the Event Ticket Tier Processor using the Hypothesis library & Pytest.

The original application had insufficient input validation, which caused runtime exceptions when unexpected inputs were provided.

Hypothesis generated a variety of test inputs and helped identify these input-handling problems. The application was then modified by introducing type checks, input format validation, exception handling & appropriate invalid-input responses.

After implementing the fixes, all 3 fuzz test functions passed successfully.

Thus, the experiment demonstrated how fuzz testing can help identify input-validation bugs and improve the reliability & robustness of software applications.